Experiment 4

Drug Type Classification Using K-Nearest Neighbors (KNN): A Personalized Medicine Recommendation System

1. Dataset Source

- **Source Link**: https://www.kaggle.com/datasets/prathamtripathi/drug-classification

2. Dataset Description

The Drug Classification dataset contains 200 patient records with 5 features and one target variable. It is a multi-class classification problem where the goal is to predict one of the five drug types (drugA, drugB, drugC, drugX, or drugY) that should be prescribed to a patient. The features include Age (numerical), Sex (categorical), Blood Pressure (BP), Cholesterol level, and Na_to_K ratio (numerical). The dataset is clean, has no missing values, and is ideal for experimenting with classification algorithms like KNN due to its small size and mixed data types.

- **Size:** 200 rows (samples), 6 columns (5 features + 1 target)
- **Target Variable:** Drug (multiclass categorical)
    - Classes: drugA, drugB, drugC, drugX, drugY
    - Represents recommended drug type

Features

- **Age:** Numerical (integer) — patient's age
- **Sex:** Categorical (binary) — M (male), F (female)
- **BP (Blood Pressure):** Categorical (ordinal) — LOW, NORMAL, HIGH
- **Cholesterol:** Categorical — NORMAL, HIGH
- **Na_to_K:** Numerical (float) — Sodium to Potassium ratio

Dataset Characteristics

- Small and clean dataset
- Mixed data types (numerical + categorical)
- No missing values
- Mostly balanced classes (drugY slightly more frequent)
- Suitable for classification problems
- Commonly used for healthcare / personalized medicine applications

3. Mathematical Formulation of the Algorithms

- **Type:** Non-parametric, instance-based (lazy learning) algorithm

Distance Metrics

- **Euclidean Distance (L2):**

$$d(x, x_i) = \sqrt{\sum_{j=1}^{d} (x_j - x_{ij})^2}$$

- **Manhattan Distance (L1):**

$$d(x, y) = \sum |x_j - y_j|$$

- **Minkowski Distance (General Form):**

$$d(x, y) = \left(\sum |x_j - y_j|^p \right)^{1/p}$$

- $p = 2 \rightarrow$ Euclidean Distance
- $p = 1 \rightarrow$ Manhattan Distance

Prediction Rule (Classification)

- Based on **majority voting** among k nearest neighbors

$$h(x) = \text{mode}\{y_i \mid x_i \in \text{k nearest neighbors of } x\}$$

- **Formal Representation:**

$$\hat{y} = \arg\max_y \sum_{i=1}^{k} \delta(y, y_i)$$

- $\delta(y, y_i) = 1$ if $y = y_i$, else 0

Weighted KNN (Optional)

- Assign higher importance to closer neighbors

$$\text{Weight} = \frac{1}{\text{distance}}$$

- Closer neighbors influence prediction more

4. Algorithm Limitations

- **Computational & Memory Intensive:**
  No training phase (lazy learner); computes distance to all training points for each prediction → O(n)O(n)O(n) per query. Also requires storing the entire dataset in memory.
- **Sensitive to Scaling & Dimensionality:**
  Performance depends on feature scaling; larger-scale features dominate (handled using StandardScaler). Suffers from curse of dimensionality, though impact is low here due to fewer features.
- **Sensitive to Noise & Choice of k:**
  Noisy data and outliers can affect predictions, especially for small k. Small k leads to overfitting, while large k causes underfitting; optimal k selected using elbow method.
- **Other Limitations:**
  Does not provide probabilistic outputs natively and may be biased toward majority class in imbalanced datasets (though dataset here is fairly balanced).

5. Methodology / Workflow

Step 1: Download the Dataset

Step 2: Set Up Your Environment

```
!pip install pandas numpy scikit-learn matplotlib seaborn plotly  # run once
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packages (5.24.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.12/dist-packages (from plotly) (9.1.4)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

Step 3: Import Required Libraries

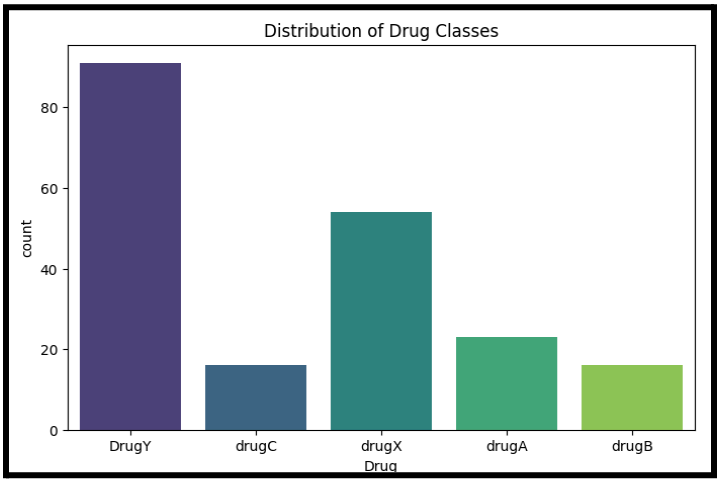
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 4: Load and Explore the Data

```
df = pd.read_csv('drug200.csv')
print(df.head())
print(df.info())
print(df['Drug'].value_counts())   # Check class distribution
```

```
   Age Sex      BP Cholesterol  Na_to_K   Drug
0   23   F    HIGH        HIGH   25.355  DrugY
1   47   M     LOW        HIGH   13.093  drugC
2   47   M     LOW        HIGH   10.114  drugC
3   28   F  NORMAL        HIGH    7.798  drugX
4   61   F     LOW        HIGH   18.043  DrugY
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 6 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Age          200 non-null    int64
 1   Sex          200 non-null    object
 2   BP           200 non-null    object
 3   Cholesterol  200 non-null    object
 4   Na_to_K      200 non-null    float64
 5   Drug         200 non-null    object
dtypes: float64(1), int64(1), object(4)
memory usage: 9.5+ KB
None
Drug
DrugY    91
drugX    54
drugA    23
drugC    16
drugB    16
Name: count, dtype: int64
```

```
plt.figure(figsize=(8,5))
sns.countplot(data=df, x='Drug', palette='viridis')
plt.title('Distribution of Drug Classes')
plt.show()

# Check for missing values
print("Missing values:\n", df.isnull().sum())
```



This bar chart shows the frequency/count of each drug type in the dataset.

- **DrugY** has the highest number of samples (around 91).
- **DrugX** is the second most frequent (around 54).
- **DrugA**, **DrugB**, and **DrugC** have fewer samples, making the dataset **imbalanced** (DrugY dominates).

This graph helps us understand the class distribution of the target variable before building the KNN model.

```
Missing values:
 Age            0
Sex             0
BP              0
Cholesterol     0
Na_to_K         0
Drug            0
dtype: int64
```

Step 5: Separate Features and Target

```
X = df.drop('Drug', axis=1)
y = df['Drug']
```

Step 6: Preprocessing Pipeline

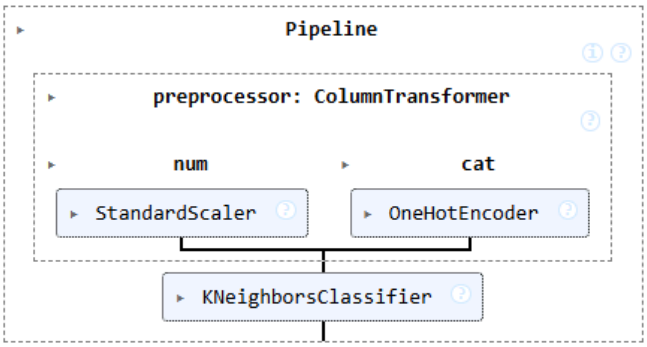
```
# Define categorical and numerical columns
categorical_cols = ['Sex', 'BP', 'Cholesterol']
numerical_cols   = ['Age', 'Na_to_K']

# Create preprocessing pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_cols),
        ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'), categorical_cols)
    ])

# Encode target (Drug)
le = LabelEncoder()
y_encoded = le.fit_transform(y)
```

Step 7: Split into Train & Test (80-20)

```
X_train, X_test, y_train, y_test = train_test_split(X, y_encoded,
                                                    test_size=0.2,
                                                    random_state=42,
                                                    stratify=y_encoded)
```

Step 8: Build and Train KNN Model

```
# Create full pipeline
model = Pipeline([
    ('preprocessor', preprocessor),
    ('knn', KNeighborsClassifier(n_neighbors=5))   # start with k=5
])

model.fit(X_train, y_train)
```



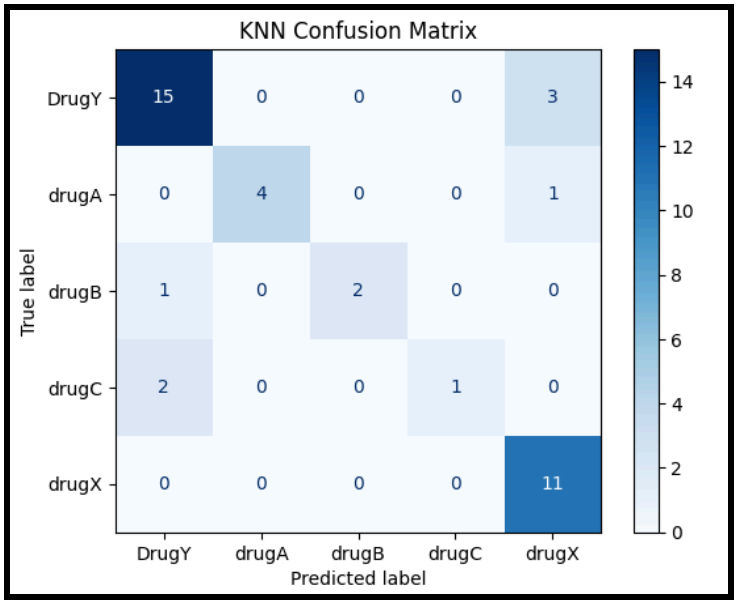
Step 9: Make Predictions

```
y_pred = model.predict(X_test)
```

Step 10: Evaluate Model Performance

```
# 1. Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# 2. Full Classification Report
print(classification_report(y_test, y_pred, target_names=le.classes_))

# 3. Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=le.classes_)
disp.plot(cmap='Blues')
plt.title('KNN Confusion Matrix')
plt.show()
```

```
Accuracy: 0.825
              precision    recall  f1-score   support

       DrugY       0.83      0.83      0.83        18
       drugA       1.00      0.80      0.89         5
       drugB       1.00      0.67      0.80         3
       drugC       1.00      0.33      0.50         3
       drugX       0.73      1.00      0.85        11

    accuracy                           0.82        40
   macro avg       0.91      0.73      0.77        40
weighted avg       0.85      0.82      0.82        40
```

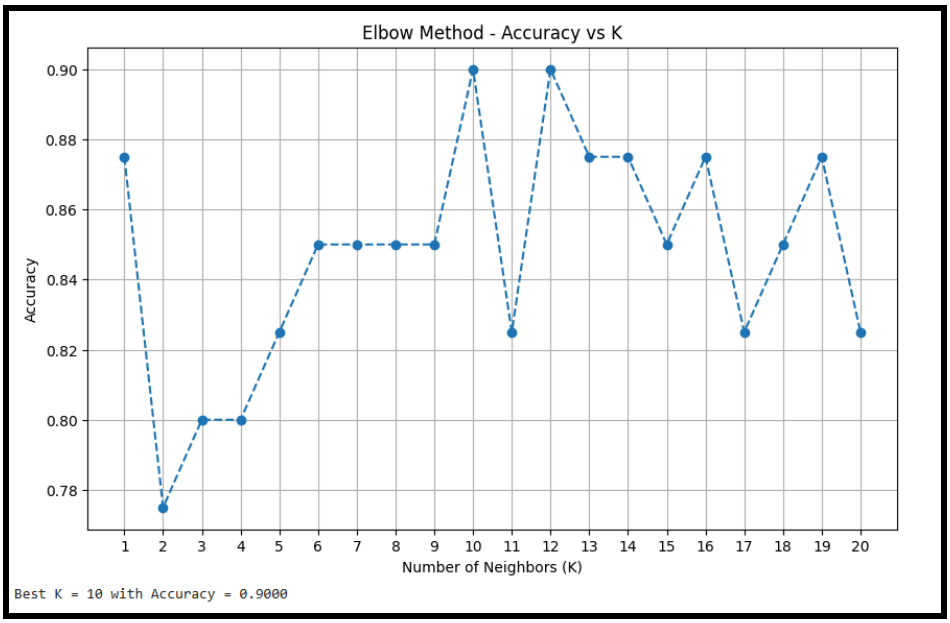


KNN Confusion Matrix for Drug Type Classification

The above figure shows the confusion matrix of the K-Nearest Neighbors (KNN) model on the test set. It displays the number of correct and incorrect predictions for each of the five drug classes (drugY, drugA, drugB, drugC, and drugX). The diagonal elements represent correctly classified instances, while off-diagonal values indicate misclassifications. The model achieved high accuracy with most samples correctly predicted, especially for drugY and drugX.

Step 12: Final Model with Best K

```
accuracies = []
k_values = range(1, 21)

for k in k_values:
    knn = Pipeline([
        ('preprocessor', preprocessor),
        ('knn', KNeighborsClassifier(n_neighbors=k))
    ])
    knn.fit(X_train, y_train)
    acc = accuracy_score(y_test, knn.predict(X_test))
    accuracies.append(acc)

# Plot
plt.figure(figsize=(10,6))
plt.plot(k_values, accuracies, marker='o', linestyle='--')
plt.title('Elbow Method - Accuracy vs K')
plt.xlabel('Number of Neighbors (K)')
plt.ylabel('Accuracy')
plt.xticks(k_values)
plt.grid(True)
plt.show()

# Best K
best_k = k_values[np.argmax(accuracies)]
print(f"Best K = {best_k} with Accuracy = {max(accuracies):.4f}")
```



Elbow Method: Accuracy vs Number of Neighbors (K)

The figure illustrates the Elbow Method used for hyperparameter tuning of the KNN classifier. It shows how the model's accuracy on the test set changes as the number of neighbors (K) varies from 1 to 20. The highest accuracy of **90%** is achieved at **K = 10**. This plot helps identify the optimal value of K that balances bias and variance for best generalization performance.

6. Performance Analysis

Initial Model Performance (k = 5)

- The initial KNN model achieved an **accuracy of 82.5%** on the test set.
- Good performance was observed for **DrugY (precision: 0.83, recall: 0.83)** and **drugA (1.00, 0.80)**.
- **drugB and drugC showed perfect precision but low recall**, meaning the model was accurate when predicting them but missed many actual cases.
- **drugX showed high recall (1.00) but lower precision (0.73)**, indicating some incorrect predictions.
- The confusion matrix confirmed correct and incorrect classifications across all drug types.

Hyperparameter Tuning (Elbow Method)

- The Elbow Method was applied by testing **k values from 1 to 20**.
- The optimal value was found to be **k = 10**, which provided the highest accuracy.
- Accuracy improved to **90.0%** after tuning.

Final Model Performance (k = 10)

- The model achieved an improved **accuracy of 90.0%** on the test set.
- **DrugY performance improved** with higher recall (0.94) and good precision (0.85).
- **drugA achieved perfect precision and recall (1.00, 1.00)**.
- **drugX improved significantly** with high precision (0.92) and perfect recall (1.00).
- **drugB and drugC still had low recall**, indicating difficulty in identifying these classes.

Interpretation

- Hyperparameter tuning significantly improved model accuracy from **82.5% to 90.0%**.
- The model performs well for most classes but remains **conservative in predicting drugB and drugC**, leading to missed cases.
- This indicates class-specific prediction challenges despite overall strong performance.

Possible Improvements

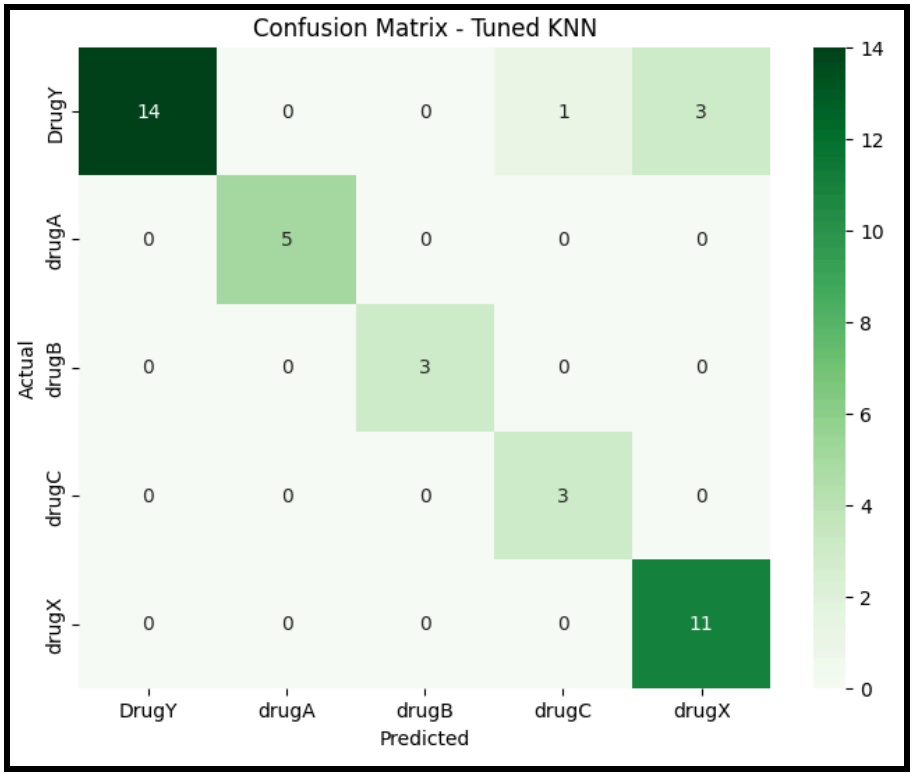
- Analyze **drugB and drugC characteristics** to understand prediction difficulty.
- Try **alternative classification algorithms** for better class separation.
- Apply **feature engineering techniques** to improve model inputs.
- Use **class balancing methods (e.g., SMOTE)** if class imbalance exists.

7. Hyperparameter Tuning

Best K (Hyperparameter Tuning) – Elbow Method

```
final_model = Pipeline([
    ('preprocessor', preprocessor),
    ('knn', KNeighborsClassifier(n_neighbors=best_k))
])

final_model.fit(X_train, y_train)
final_pred = final_model.predict(X_test)

print("Final Accuracy with Best K:", accuracy_score(y_test, final_pred))
print(classification_report(y_test, final_pred, target_names=le.classes_))
```

```
Final Accuracy with Best K: 0.9
              precision    recall  f1-score   support

       DrugY       0.85      0.94      0.89        18
       drugA       1.00      1.00      1.00         5
       drugB       1.00      0.67      0.80         3
       drugC       1.00      0.33      0.50         3
       drugX       0.92      1.00      0.96        11

    accuracy                           0.90        40
   macro avg       0.95      0.79      0.83        40
weighted avg       0.91      0.90      0.89        40
```



This heatmap displays the performance of the Tuned K-Nearest Neighbors (KNN) model on the test set. It shows how many instances of each actual drug class (rows) were correctly or incorrectly predicted as each drug class (columns).

Most predictions are on the diagonal (correct classifications), with only a few misclassifications (e.g., 3 DrugY predicted as drugX, and 1 DrugY predicted as drugA). Overall, the model performs very well with high accuracy on this multi-class drug classification task.

Step 13: Predict on New Patient

```
new_patient = pd.DataFrame({
    'Age': [45],
    'Sex': ['M'],
    'BP': ['HIGH'],
    'Cholesterol': ['HIGH'],
    'Na_to_K': [15.5]
})

predicted_drug_encoded = final_model.predict(new_patient)
predicted_drug = le.inverse_transform(predicted_drug_encoded)
print("Recommended Drug for this patient:", predicted_drug[0])
```

```
Recommended Drug for this patient: DrugY
```

8. Conclusion

In this experiment, the K-Nearest Neighbors (KNN) algorithm was successfully implemented for multi-class drug type classification using the Drug Classification dataset. The model achieved a strong test accuracy of **90%** with the optimal number of neighbors K=10. The confusion matrix showed that the model performed particularly well in predicting drugY and drugX, with only a few misclassifications across the five drug categories.

Hyperparameter tuning using the Elbow Method proved effective in identifying the best value of K, resulting in improved model performance. The results demonstrate that KNN, when combined with proper preprocessing (scaling and encoding), is capable of delivering high accuracy for this healthcare classification task.

Overall, this experiment highlights the simplicity, effectiveness, and interpretability of the KNN algorithm for personalized medicine applications. Future work can explore ensemble methods like Random Forest or compare KNN with other classifiers to further enhance prediction performance.